HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND EDUCATION



PROJECT

SMARTVEST-SAR: AN IOT AND AI-DRIVEN LIFE-SAVING SYSTEM FOR MARITIME RESCUE

Project Mentor: Associate Professor Dr. Le My Ha

Student: Thakorn Tangphadungrutch (Le Duc Tri)

HO CHI MINH CITY, September 2026



PROJECT

SMARTVEST-SAR: AN IOT AND AI-DRIVEN LIFE-SAVING SYSTEM FOR MARITIME RESCUE

Project Mentor: Associate Professor Dr. Le My Ha

Student: Thakorn Tangphadungrutch (Le Duc Tri)

PROJECT POSITIONING

The supplied project package includes a physical prototype photograph, embedded firmware for the life-jacket node, firmware for an intermediate LoRa relay, and a Python desktop application. These materials support a coherent system narrative. However, they do not by themselves verify an 8 km radio range, multi-hop coverage of tens or hundreds of kilometres, continuous solar energy autonomy, or prediction accuracy at sea. Those items are therefore described as design targets or capabilities to be validated rather than as established experimental results.

A particularly important technical distinction concerns the project's "AI" wording. The current desktop code does not contain a trained machine-learning model. Instead, it uses a transparent wind-informed heuristic that combines recent GPS movement direction with current and recent wind direction. In this report, the feature is called AI-assisted or data-driven only at the application level; the implementation is described precisely as a heuristic drift predictor. This makes the portfolio more credible and gives a clear path for a future machine-learning upgrade.

Before submitting this report with an application, replace the bracketed student/mentor fields and include only contributions personally completed by the student. A short contribution statement is provided in Appendix C.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Rescue Problem and Engineering Motivation 1
1.2 Project Objectives 1
1.3 Scope, Prototype Positioning, and Claims Boundary 1
1.4 Engineering Workflow 2
1.5 Report Structure 2
CHAPTER 2: FROM RESCUE NEED TO ENGINEERING DESIGN 3
2.1 Target Use Case and Engineering Requirements 3
2.2 Overall System Architecture 3
2.3 GPS Positioning and Data Representation 4
2.4 LoRa Communication and Multi-hop Relay Concept 5
2.5 Temperature Monitoring and Heating Function 5
2.6 Solar-assisted Power and Lithium Storage 6
2.7 Rescue-station Software 6
2.8 Drift Prediction Logic and the Use of “AI” 6
2.9 Safety, Reliability, and Human-use Constraints 7
2.10 Design Rationale Summary 8
CHAPTER 3: PROTOTYPE IMPLEMENTATION 9
3.1 Physical Prototype Overview 9
3.2 Main Wearable Controller and Firmware 10
3.3 GPS Acquisition and Packet Buffering 12
3.4 LoRa Acknowledgement and Retry Protocol 12
3.5 Intermediate Relay Buoy Firmware 12
3.6 Thermal-control Implementation 13
3.7 Solar and Battery Integration 14
3.8 USB LoRa Receiver and Desktop Application 14
3.9 Drift Prediction Algorithm in the Current Software 14
3.10 Implementation Parameters 14
3.11 Current Prototype Limitations 17

CHAPTER 4: PROTOTYPE TESTING AND VALIDATION 18
4.1 Validation Goals 18
4.2 Recommended Test Setup 18
4.3 Validation Matrix 19
4.4 GPS Verification 19
4.5 Point-to-point LoRa Verification 19
4.6 Relay-network Verification 19
4.7 Thermal and Power Verification 20
4.8 Software and Prediction Verification 20
4.9 Evidence Available from the Current Project Package 21
4.10 Evidence Recommended for a College Portfolio 21
CHAPTER 5: DISCUSSION, CONCLUSION AND FUTURE DEVELOPMENT .. 23
5.1 What the Prototype Demonstrates 23
5.2 Strengths and Engineering Trade-offs 23
5.3 Limitations 23
5.4 Future Development 24
5.5 Conclusion 24
APPENDIX A: SOURCE-CODE TRACEABILITY 26
APPENDIX B: FINAL DEMONSTRATION CHECKLIST 28
APPENDIX C: COLLEGE-APPLICATION PORTFOLIO SUMMARY 29

LIST OF FIGURES

Figure 2.1. Overall architecture of the smart life-jacket system 4
Figure 2.2. Conceptual store-and-forward LoRa relay network 5
Figure 2.3. Power architecture for solar-assisted operation 6
Figure 2.4. Current software drift-prediction workflow 7
Figure 3.1. Annotated photograph of the integrated smart life-jacket prototype 9
Figure 3.2. Main life-jacket firmware flow derived from the provided source code . 11
Figure 3.3. Intermediate relay-buoy firmware flow derived from the provided source code 13
Figure 3.4. Bench-test and intended field-reporting timing concept 16
Figure 4.1. Staged prototype validation strategy 18
Figure 4.2. Illustrative received trajectory and drift prediction; not field-test data . 20
Figure 4.3. Recommended evidence package for a college engineering portfolio 22

LIST OF TABLES

Table 2.1. Rescue problem-to-engineering requirement traceability 3
Table 2.2. Communication architecture and design status 5
Table 2.3. Safety and reliability requirements for the prototype 7
Table 3.1. Main prototype subsystems and source-supported implementation 10
Table 3.2. Key implementation parameters extracted from the supplied code 15
Table 4.1. Recommended prototype validation matrix 19
Table 4.2. Quantitative metrics recommended for field testing 21
Table 5.1. Main strengths and engineering trade-offs 23
Table A.1. Source-code traceability to report functions 26

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACK	Acknowledgement
AI	Artificial Intelligence
API	Application Programming Interface
ESP32	Espressif 32-bit microcontroller family
GPS	Global Positioning System
GUI	Graphical User Interface
Li-ion	Lithium-ion battery
LoRa	Long Range low-power radio modulation / communication technology
MCU	Microcontroller Unit
PDR	Packet Delivery Ratio
RSSI	Received Signal Strength Indicator
SOC	State of Charge
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus

CHAPTER 1: INTRODUCTION

1.1 Rescue Problem and Engineering Motivation

A person separated from a vessel or drifting at sea can be difficult to relocate because the search area grows with time and because the survivor may move continuously under the combined influence of current, wind, and waves. A conventional life jacket provides flotation, but it does not normally provide an electronic record of the wearer's position, a communication path beyond visual range, or a way to estimate where the wearer may drift next. The project uses this practical rescue problem as the motivation for an application-focused engineering prototype.

The design question is therefore: can a wearable life-jacket platform periodically determine its GPS position, transmit the position through a long-range LoRa radio link, extend coverage through intermediate relay buoys, provide limited thermal support in low-temperature conditions, and present the received track to rescuers together with an estimate of the next drift direction? The objective is not to replace maritime emergency beacons or certified rescue equipment. It is to demonstrate how multiple engineering subsystems can be integrated into one student-built prototype.

1.2 Project Objectives

The main objective is to design and implement a proof-of-concept smart life jacket that connects wearable hardware, embedded firmware, radio communication, relay networking, renewable-energy support, and computer software into one understandable rescue workflow.

- Acquire the wearer's geographic position continuously from a GPS receiver and store recent position packets for transmission.
- Transmit life-jacket position data over LoRa and use acknowledgement/retry logic so that an unacknowledged packet is retained rather than immediately discarded.
- Use intermediate LoRa relay nodes to receive, de-duplicate, buffer, and forward packets toward a rescue station when a direct radio link is unavailable.
- Monitor temperature and enable a heating output below a configured threshold in the current prototype firmware.
- Use a solar panel together with lithium energy storage to support the electronic load and reduce dependence on a one-time battery charge.
- Receive LoRa data at a computer through a USB/serial interface, visualize received GPS points, obtain wind information, and estimate a likely short-term drift direction.
- Define staged validation tests and limitations so the project can be presented as an engineering portfolio rather than as unverified rescue-product claims.

1.3 Scope, Prototype Positioning, and Claims Boundary

This project is deliberately positioned as an educational prototype. The physical device shown in the supplied photograph demonstrates integration of a life-jacket body, buoyant structural members, an electronics enclosure with antenna, an attached solar panel, and supporting modules/wiring. The embedded source code demonstrates GPS parsing, thermal-threshold logic, LoRa packet transmission, acknowledgement handling, and relay forwarding.

The desktop code demonstrates serial reception, map visualization, weather-data retrieval, and short-horizon drift extrapolation.

Several performance statements remain design targets rather than verified results in the supplied package. The project description specifies a nominal LoRa hop radius of approximately 8 km and an intended five-minute reporting interval. The firmware provided for the wearable node currently uses a 5,000 ms GPS-update timer for bench testing, which is 5 seconds rather than 5 minutes. Likewise, the code implements store-and-forward relaying but does not implement a complete self-routing mesh protocol with route discovery, link-quality optimization, or automatic topology management. These differences are documented rather than hidden because accurate boundary-setting is an important engineering skill.

1.4 Engineering Workflow

1. Define the rescue scenario and identify what information rescuers actually need: current position, recent track, communication status, and a plausible next drift direction.
2. Convert the scenario into engineering requirements for positioning, communication interval, radio reach, energy use, thermal behavior, physical integration, and software output.
3. Design the wearable node: GPS, temperature sensing, microcontroller, LoRa radio, heater control, battery, solar charging, and enclosure.
4. Design the communication logic: packet format, queue, acknowledgement, retry, relay buffering, and duplicate suppression.
5. Build the rescue-station software for serial reception, mapping, environmental-data input, and trajectory prediction.
6. Verify each subsystem independently before conducting an integrated dry test and then controlled outdoor/field tests.
7. Collect evidence that can be explained by the student: photographs, code, circuit information, range data, power measurements, packet logs, and prediction-error plots.

1.5 Report Structure

Chapter 2 translates the rescue need into engineering design decisions and explains the role of each subsystem. Chapter 3 documents the actual prototype implementation derived from the supplied photograph and source code. Chapter 4 defines a staged verification plan and separates currently available evidence from tests that should still be documented. Chapter 5 discusses the prototype’s strengths, trade-offs, limitations, and future development. The appendices provide source-code traceability, a final demonstration checklist, and a concise college-application portfolio summary.

CHAPTER 2: FROM RESCUE NEED TO ENGINEERING DESIGN

2.1 Target Use Case and Engineering Requirements

The target use case is a person wearing a life jacket after separation from a vessel in an offshore or coastal environment. The system should continue producing useful position information even when rescuers do not have visual contact. Because a wearable rescue device has severe constraints on size, power, waterproofing, antenna placement, thermal safety, and user comfort, the design is treated as a layered system rather than as a single “smart” module.

Rescue / design issue	Engineering requirement	Prototype design decision	Verification method
Survivor drifts after separation	Track position repeatedly over time	GPS receiver connected to the wearable controller	Compare reported coordinates with a trusted reference during stationary and moving tests
Direct radio link may be unavailable	Retain and forward packets through intermediate nodes	LoRa store-and-forward relay with packet buffer and ACK handling	Point-to-point and two-hop packet-delivery tests
Power is limited	Reduce dependency on one battery charge	Solar-assisted charging plus lithium storage	Measure current draw, charge current and runtime under defined duty cycles
Cold conditions can increase thermal stress	Provide bounded thermal support only under safe conditions	Temperature-threshold heater output in prototype firmware	Bench temperature test with independent thermometer and surface-temperature limit
Rescuers need more than the latest point	Show track and likely short-term movement	Desktop map and drift prediction using recent GPS history and wind data	Replay logged routes and compare predicted positions with later measured positions
Wearable electronics face water and impact	Protect electronics and avoid exposed hazards	Enclosure, secured wiring, buoyant structure and conservative operating states	Dry inspection, splash/waterproof tests appropriate to the prototype, strain checks

Table 2.1. Rescue problem-to-engineering requirement traceability

2.2 Overall System Architecture

The system is organized into three layers: the wearable life-jacket node, one or more relay nodes, and the rescue-station computer. The wearable node produces sensor data and radio packets. Relay nodes extend the communication path by buffering and forwarding unique packets. The rescue station acknowledges received packets, stores the history, displays the route, and computes a short-term drift estimate. This separation makes the project easier to explain and test because each layer has a specific responsibility.

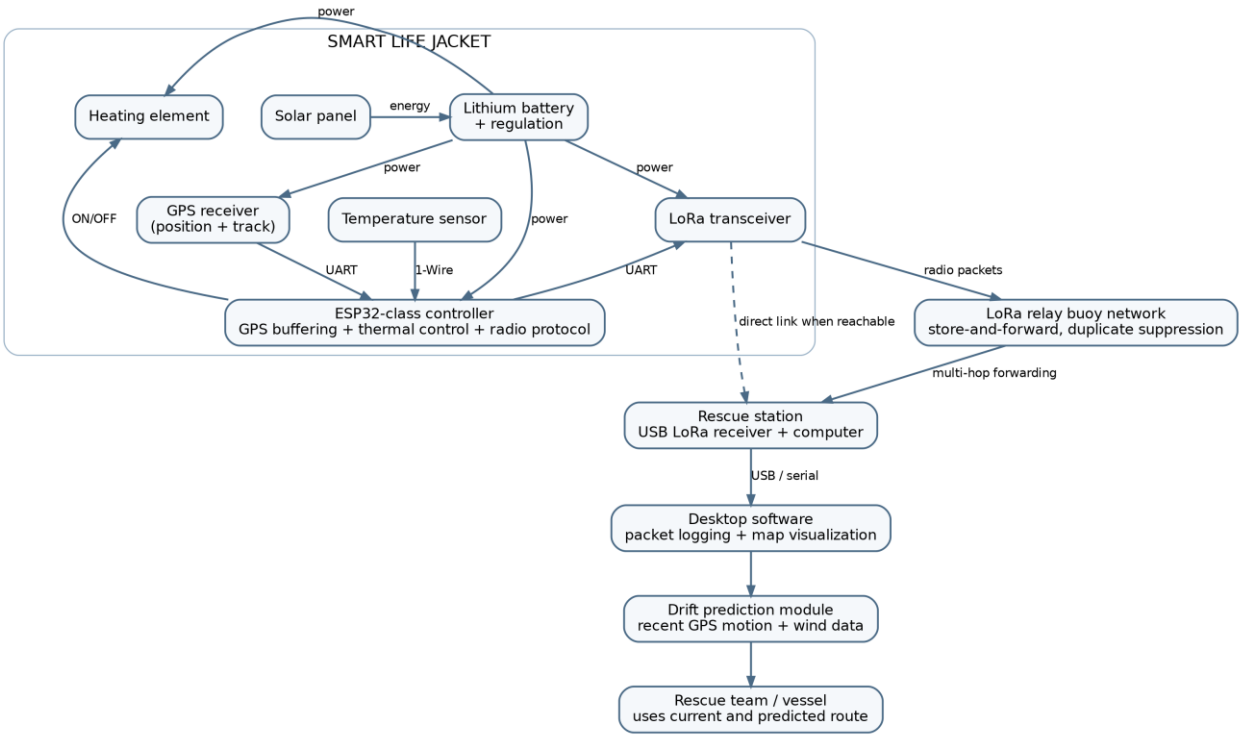


Figure 2.1. Overall architecture of the smart life-jacket system

Figure interpretation. The key design idea is to keep the rescue information path separate from the power path. GPS and temperature data enter the embedded controller; position packets move through LoRa links toward the rescue station; solar energy charges a stored-energy subsystem; and the heating element is controlled as a separate high-current load. The rescue-station software uses received positions and environmental wind data to assist route planning.

2.3 GPS Positioning and Data Representation

The wearable firmware uses the TinyGPS++ library and reads a GPS receiver on a dedicated serial interface configured at 38,400 bit/s. When the GPS library reports an updated location, the firmware stores latitude and longitude and creates a simple text packet. In the current source code the packet format is “1-latitude,longitude”, where the leading value acts as an identifier and the coordinates are formatted to four decimal places.

```
a[sl] = "1-" + String(lat,4) + "," + String(lon,4);
// Example conceptual packet:
1-10.8231,106.6297
```

A text packet is appropriate for a first prototype because it is easy to inspect in the serial monitor and easy to parse on the desktop computer. For a later field version, the packet should include at least a device ID, sequence number, timestamp, latitude, longitude, battery state, temperature, and an integrity check. A sequence number is especially useful in a relay network because it provides a robust way to detect duplicates and out-of-order packets.

2.4 LoRa Communication and Multi-hop Relay Concept

LoRa is selected because the project requires low-data-rate communication over distances much longer than common short-range wireless links. The current design sends compact position strings rather than images or high-bandwidth telemetry. The project description uses an approximately 8 km hop as the design target. Actual range depends on frequency band, transmit power, antenna gain and height, receiver sensitivity, regulatory limits, sea state, obstruction, interference, enclosure loss, and the placement of both ends of the link. Therefore, the hop range should be demonstrated experimentally rather than inferred from the radio technology alone.

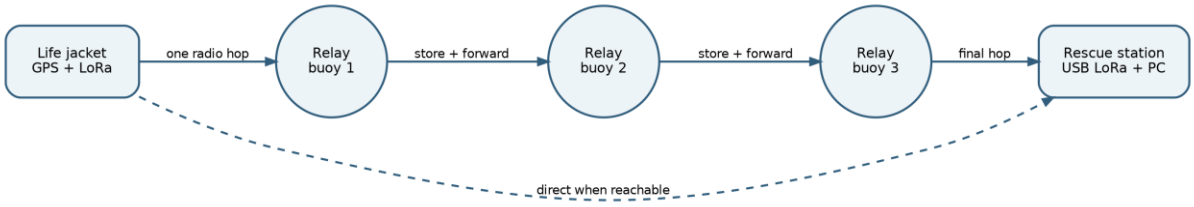


Figure 2.2. Conceptual store-and-forward LoRa relay network

Figure interpretation. Each intermediate node can store a unique packet and forward it toward the next reachable node. Conceptually, several well-positioned relays can extend the end-to-end communication corridor well beyond a single hop. Coverage is not guaranteed to increase by exactly 8 km per node because real radio links depend on geometry, antenna height, line of sight, interference, and link margin.

Communication element	Current implementation	Strength	Boundary / next step
Wearable transmitter	UART-connected LoRa module; sends oldest buffered packet	Simple, low data volume, easy debugging	Exact radio model/frequency/link budget not provided
Acknowledgement	Receiver returns text token “ok”	Prevents acknowledged packet from remaining forever	Needs sequence numbers and timeout statistics for robust field use
Retry	Unacknowledged packet remains in queue	Tolerates temporary packet loss	Repeated immediate retries may increase energy use
Relay node	Stores unique strings, forwards oldest packet, removes after ACK	Implements basic store-and-forward behavior	No route discovery, hop count, TTL, link-quality routing, or network-wide addressing
End station	USB/serial receiver handled by Python app	Integrates communication and visualization	Should log timestamps, RSSI/SNR if available, and packet sequence numbers

Table 2.2. Communication architecture and design status

2.5 Temperature Monitoring and Heating Function

The wearable firmware initializes a 1-Wire temperature sensor through the DallasTemperature library and controls a digital heating output. In the supplied code, the heater output is set HIGH when the measured temperature is below 25 °C and LOW otherwise. This demonstrates the sensing-to-actuation logic clearly, but the 25 °C threshold should be understood as a prototype software setting rather than a medically validated or maritime-survival threshold.

For any future human-worn test, thermal safety is more important than maximum heating. The system should measure the temperature at the relevant location, bound heater surface temperature, use electrical isolation/protection appropriate to the environment, include a manual shutoff, and enter a safe state if the sensor is disconnected or produces invalid data. The present firmware checks for a disconnected sensor value before updating its stored temperature but should be extended with an explicit fault state that forces the heater OFF.

2.6 Solar-assisted Power and Lithium Storage

The physical prototype photograph shows a small solar panel, and the project description specifies lithium energy storage. The engineering role of the solar subsystem is to extend operating time and replenish stored energy when irradiance is available. Without measured panel output, battery capacity, regulator efficiency, radio duty cycle, GPS consumption, and heater current, the system should be described as solar-assisted rather than continuously solar-powered.

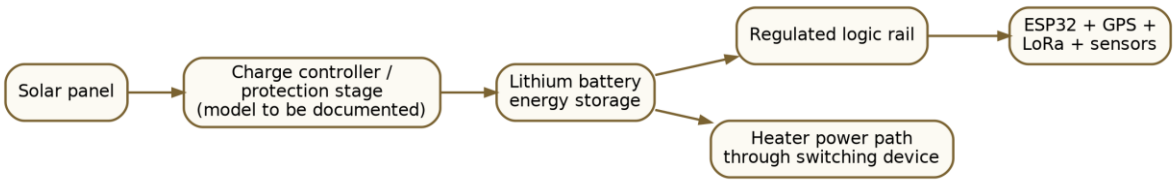


Figure 2.3. Power architecture for solar-assisted operation

Figure interpretation. The solar panel should feed a charging/protection stage before the lithium battery. A regulated logic rail should supply the microcontroller, GPS, LoRa radio and sensors, while the heater should use a separately protected switching path sized for its current. The exact battery, charger and regulator models were not provided in the source package and should be documented in the bill of materials before final submission.

2.7 Rescue-station Software

The desktop application is written in Python with Tkinter, TkinterMapView, Pillow, pyserial, threading, requests, and standard mathematical functions. It enumerates serial ports, opens the selected receiver at 9,600 bit/s, reads incoming text packets in a background thread, immediately returns an “ok” acknowledgement, stores each unique packet, and plots received positions on an interactive map.

For every newly accepted GPS point, the program queries the Open-Meteo forecast API for current wind speed and wind direction at the received coordinates. This environmental input is combined with recent movement history when the user requests map display. The resulting

software therefore demonstrates a complete data path from a radio receiver to a rescue-oriented visual interface.

2.8 Drift Prediction Logic and the Use of "AI"

The current code implements a lightweight heuristic predictor rather than a trained machine-learning model. This is a positive feature for a first prototype because the logic is transparent: a student can explain exactly how the next direction is generated. The most recent movement direction is estimated from the last two received GPS points. A weighted circular average is calculated from up to five recent wind directions, with newer entries receiving larger weights. The code then blends the movement direction, recent wind average, and current wind direction.

In simplified form, the target direction used by the current implementation is

$$\theta_{target} = 0.70\,\theta_{move} + 0.20\,\theta_{wind,avg} + 0.10\,\theta_{wind,now}$$

The predictor does not instantaneously rotate to the target direction. It changes the heading gradually with a turn-rate factor that increases with wind speed and is capped in the code. It then generates 20 future markers while slightly decreasing step length. These future markers are a visualization of a plausible short-term path, not a validated ocean-drift model.

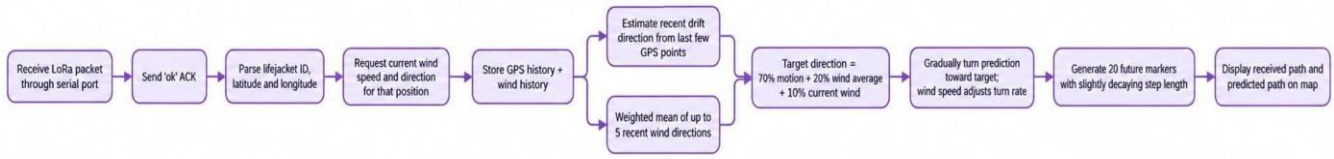


Figure 2.4. Current software drift-prediction workflow

Figure interpretation. The algorithm is explainable and suitable for an application prototype, but it should not be advertised as a learned AI model. A future AI version could be trained from real trajectories using GPS history, wind, current, wave, and possibly vessel/region data, and then compared against the transparent heuristic baseline.

2.9 Safety, Reliability, and Human-use Constraints

Risk / issue	Prototype requirement	Recommended implementation / response
Heater overheating	Never rely only on one software threshold	Independent temperature limit, fuse/current limit, fail-OFF behavior, manual disconnect
Battery short / water ingress	Prevent thermal/electrical hazard	Protected cells, sealed enclosure, strain relief, corrosion-resistant connectors, splash/immersion test appropriate to prototype
GPS data stale or invalid	Do not present old data as current	Timestamp every fix and mark age/status in software
Radio packet loss	Retain data and make loss observable	Sequence numbers, ACK, bounded retry, packet-delivery logging, RSSI/SNR logging when supported
Relay loop / duplicate packet	Prevent uncontrolled circulation	Unique packet identifier, duplicate cache, hop limit / TTL
Solar power insufficient	Do not claim continuous operation without an energy budget	Measure worst-case current and realistic solar input; state expected runtime
Unverified rescue performance	Avoid representing the prototype as certified safety equipment	Controlled demonstrations only; no replacement for approved maritime distress systems

Table 2.3. Safety and reliability requirements for the prototype

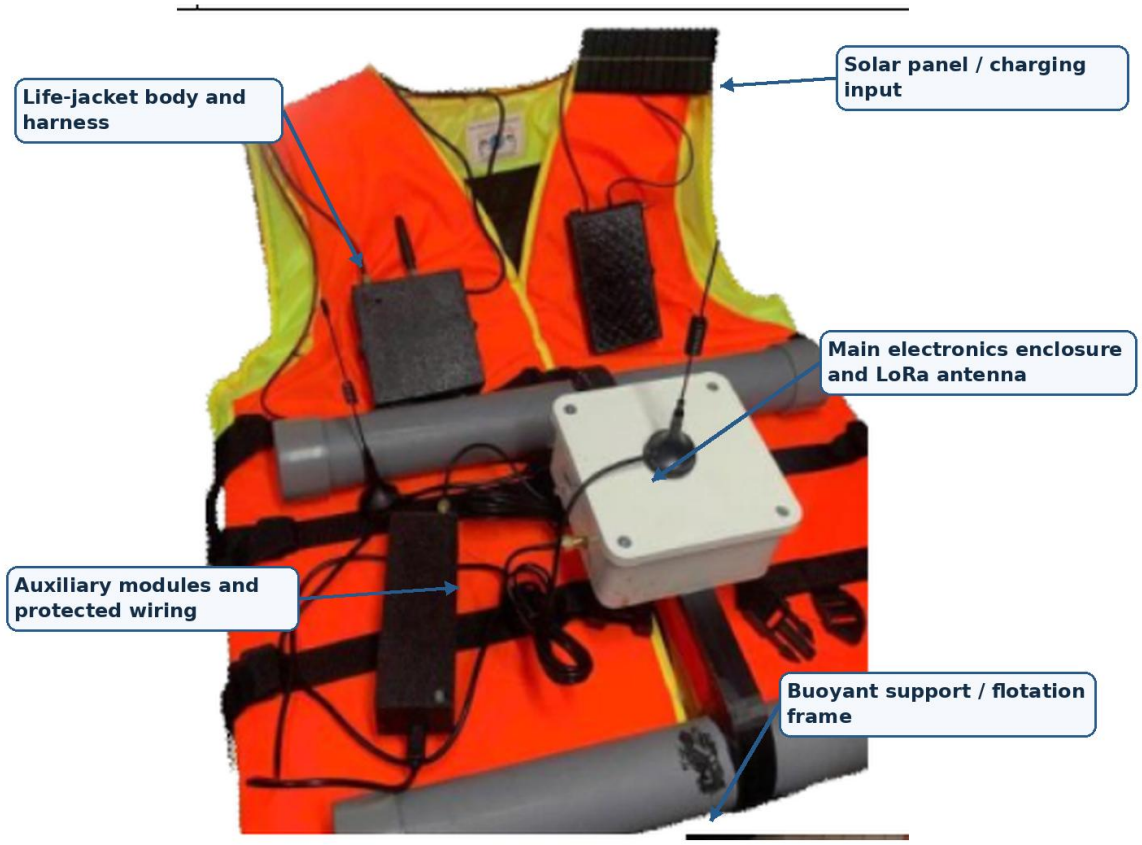
2.10 Design Rationale Summary

The design is a sensible student-level compromise: GPS provides understandable position data; LoRa supports low-rate long-distance communication; store-and-forward relays demonstrate network extension without requiring cellular infrastructure; temperature sensing connects an environmental measurement to an actuator; solar charging introduces energy engineering; and the Python application turns raw packets into a map and drift estimate. Most importantly, the project is explainable from end to end, which is exactly what makes it valuable in a college engineering portfolio.

CHAPTER 3: PROTOTYPE IMPLEMENTATION

3.1 Physical Prototype Overview

The supplied photograph shows the electronics integrated onto a high-visibility life-jacket structure. Visible features include a small solar panel near the upper portion of the vest, a rigid electronics enclosure with an external antenna, additional black modules and wiring secured to the jacket, and grey buoyant support members. The physical integration demonstrates that the project progressed beyond isolated breadboard code and toward a wearable prototype configuration.



Prototype photograph supplied by the project team. Labels identify visible subsystem locations without assuming unprovided component

Figure 3.1. Annotated photograph of the integrated smart life-jacket prototype

Figure interpretation. The photograph supports the report's claim of physical subsystem integration. The labels intentionally avoid naming specific component models that were not supplied. Before a final college submission, the student should add one or two higher-resolution photographs showing the front, rear, electronics enclosure interior, and solar/battery arrangement.

Subsystem	Source-supported implementation	Role

Subsystem	Source-supported implementation	Role
Wearable platform	Life-jacket body with integrated modules, enclosure, antenna, wiring and buoyant structure visible in photograph	Mechanical platform and flotation support
Controller	Embedded firmware written for an ESP32-style platform with multiple hardware serial interfaces	Sensor acquisition, packet buffering, radio protocol and heater control
Position sensing	TinyGPS++ parsing through Serial2 at 38,400 bit/s	Latitude/longitude acquisition
Temperature sensing	OneWire + DallasTemperature library on pin 21	Temperature monitoring
Heating output	Digital output on pin 25	Turns heater output on/off according to threshold logic
LoRa link	Serial1 at 9,600 bit/s on wearable and relay firmware	Packet transmission and acknowledgement handling
Relay node	Separate firmware with queue, duplicate check and forwarding	Store-and-forward multi-hop extension
Rescue software	Python/Tkinter application with serial reception, map display and Open-Meteo requests	Visualization, environmental context and drift prediction
Solar + battery	Described by project team and solar panel visible in photograph	Energy harvesting and storage; detailed BOM not supplied

Table 3.1. Main prototype subsystems and source-supported implementation

3.2 Main Wearable Controller and Firmware

The wearable firmware initializes three main hardware interfaces: a 1-Wire bus for the temperature sensor, a serial port for GPS, and a serial port for LoRa. During each loop it updates temperature, periodically checks GPS, and then attempts to transmit the oldest packet in its queue. The use of a queue is important because position data are not immediately discarded when the radio path is unavailable.

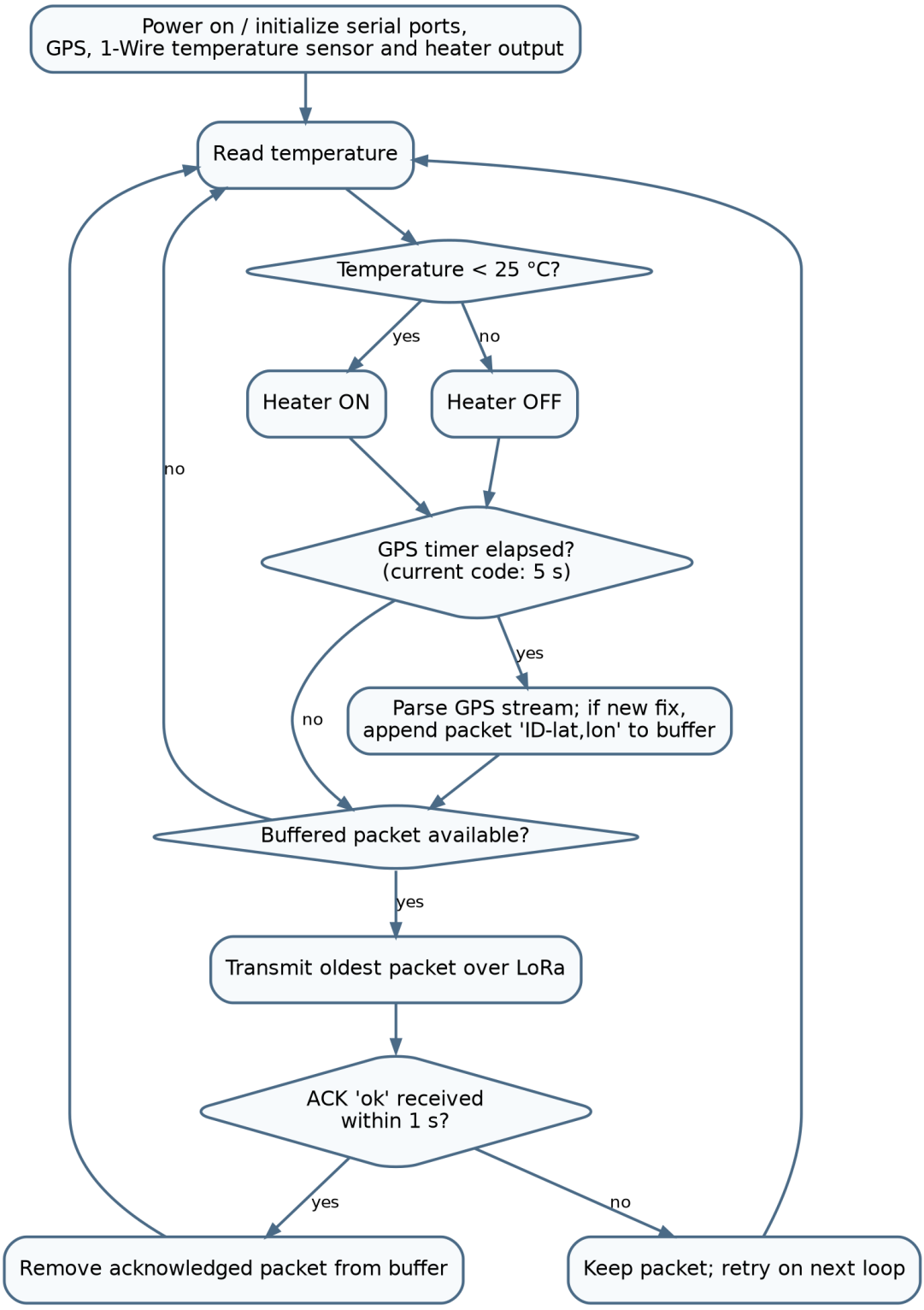


Figure 3.2. Main life-jacket firmware flow derived from the provided source code

Figure interpretation. The current loop prioritizes simple, visible behavior. Temperature is checked every loop, GPS is refreshed on a timer, new fixes are buffered, and the oldest buffer entry is transmitted until an acknowledgement is received. For field use, this flow should be converted to non-blocking timing so radio waiting and temperature conversion do not delay other functions.

3.3 GPS Acquisition and Packet Buffering

The firmware reads raw GPS bytes whenever they are available and passes them to TinyGPS++. If the library reports an updated fix, the program copies latitude and longitude to local variables, increments the queue count, and stores a formatted packet. The source declares a String array with capacity for 5,000 entries. This is convenient for a prototype but is memory-inefficient on a microcontroller; a fixed-size ring buffer of compact records would be safer in a long-duration version.

The current code formats latitude and longitude to four decimal places. This simplifies packet length but introduces coordinate quantization at approximately the ten-metre scale in latitude before normal GPS measurement uncertainty is considered. A future version can preserve five or six decimal places if the radio link budget and packet format permit it.

3.4 LoRa Acknowledgement and Retry Protocol

If at least one packet is buffered, the wearable node transmits the oldest packet and listens for up to one second. When the incoming line equals “ok”, the program removes that packet by shifting the remaining queue. If no acknowledgement is received, the packet stays in the queue and is transmitted again during a later loop. This mechanism gives the prototype a basic form of reliability over an unreliable radio channel.

```
if(sl > 0){
  lora.println(a[1]);
  ...
  if(data == "ok"){
    remove_gps();
  }
}
```

For the next version, each packet should carry a sequence number and timestamp so that the receiver can acknowledge a specific packet rather than any line of text. Retry attempts should be rate-limited, and the software should record failed attempts. These changes would make packet-delivery performance measurable.

3.5 Intermediate Relay Buoy Firmware

The intermediate relay firmware listens for LoRa messages, stores a received message only if it is not already in its queue, flashes an indicator LED, and then forwards the oldest queued message. When an “ok” acknowledgement is received, it removes the acknowledged message. If another non-acknowledgement packet arrives while waiting, the relay can add that packet to its own queue. This is a compact demonstration of duplicate suppression and store-and-forward networking.

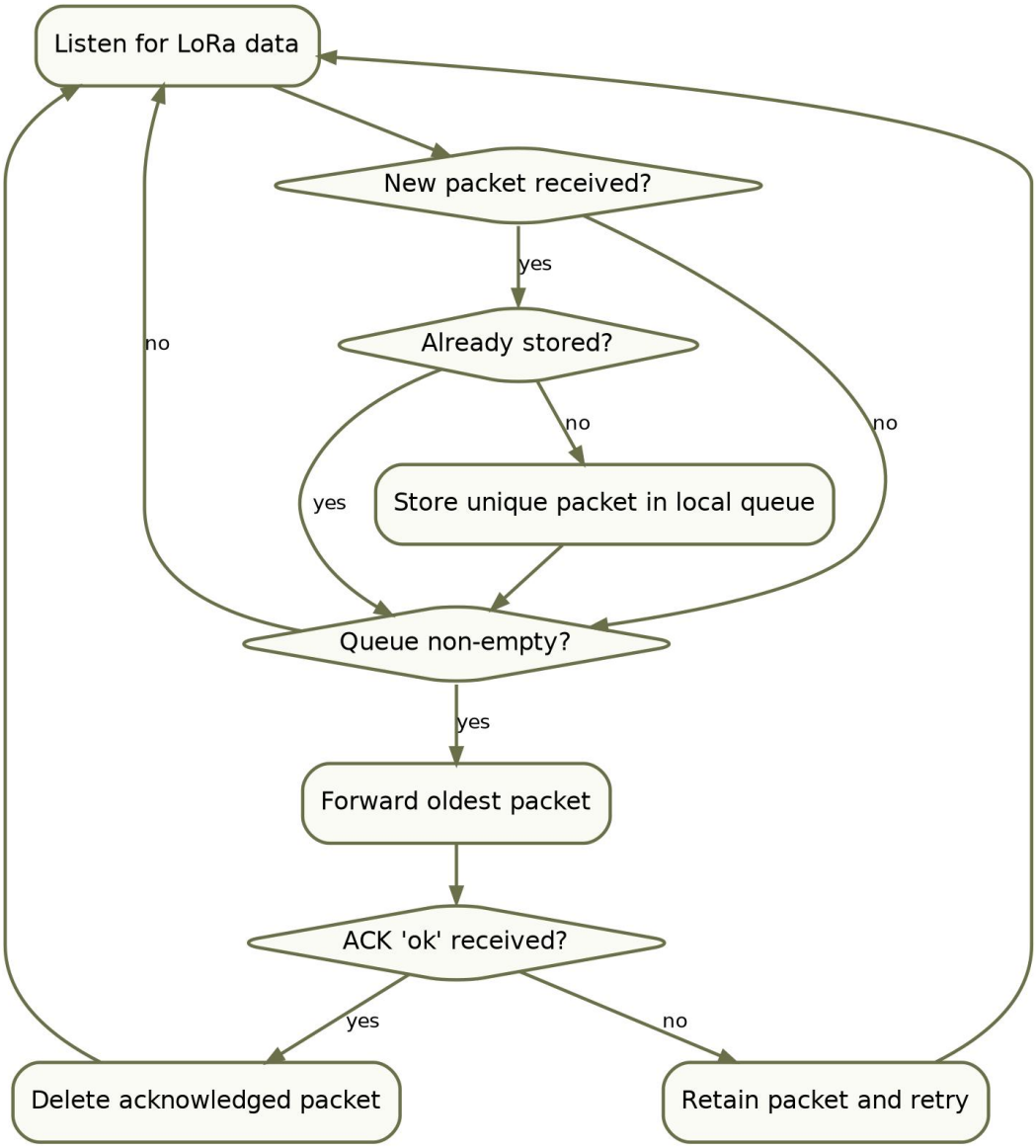


Figure 3.3. Intermediate relay-buoy firmware flow derived from the provided source code

Figure interpretation. The relay does not know a complete network topology; it simply remembers unique packet strings and forwards them. The mechanism is useful for a line or sparse relay chain, but a full mesh version would require node addressing, route selection or flooding rules, hop limits, and stronger duplicate identification.

3.6 Thermal-control Implementation

The thermal-control function calls the temperature sensor, stores the measured value when valid, and uses a simple threshold: below 25 °C the heater output is enabled; at or above 25 °C it is disabled. This is appropriate as a first demonstration of sensor-triggered actuation. Because the heater is attached to a wearable rescue device, a future version should add hysteresis, maximum-on time, an independent over-temperature cutout, and explicit fault handling that forces the output OFF if sensor data are stale or invalid.

3.7 Solar and Battery Integration

The source code does not contain battery-state measurement or solar charge-control logic; these functions are therefore treated as hardware-level subsystems. The project photograph and project description support the presence of a solar panel and lithium storage, but the exact panel rating, cell configuration, charger, protection board, regulator, and heater current were not supplied. A final bill of materials should record these details because they determine runtime and safety.

A useful engineering calculation for the portfolio is an energy budget: measure average current for GPS tracking, LoRa idle, LoRa transmission, controller, display/indicators if any, and heater duty cycle. Compare daily energy consumption with measured daily solar energy under realistic conditions. This turns the statement “powered by solar” into a quantitative design argument.

3.8 USB LoRa Receiver and Desktop Application

The Python application scans available COM ports and allows the operator to connect or disconnect from a selected serial receiver. A background thread continuously reads serial data so the GUI remains responsive. Every non-empty line is acknowledged with “ok”, displayed in a text area, checked for duplication, parsed into a device identifier and coordinates, and stored for later visualization.

The map display uses a blue/normal arrow icon for received points. The latest position becomes the map centre and the application sets a local zoom level. Because the application is designed for a rescue operator rather than only for debugging, the next useful features would be automatic timestamp display, age-of-last-fix warning, device status, packet count, battery information, relay-hop information, and an export button for CSV/KML logs.

3.9 Drift Prediction Algorithm in the Current Software

When at least two GPS positions are available, the program calculates a recent movement vector from the last two points. It also stores wind direction and wind speed obtained from Open-Meteo for each accepted position. Up to five recent wind directions are averaged using sine/cosine components and linearly increasing weights. The target direction is formed from 70% recent movement, 20% weighted wind average, and 10% current wind direction.

Wind speed influences how quickly the predicted path is allowed to turn. In the supplied code, the per-step turn-rate term is $0.3 + 0.04 \times$ wind speed and is capped at 2.0. The program then generates 20 future markers; the step length is multiplied by 0.995 at every prediction step. These numbers are software design choices rather than learned parameters or values validated against oceanographic data.

3.10 Implementation Parameters

Parameter	Value in supplied implementation	Engineering interpretation
Wearable GPS serial rate	38,400 bit/s	Configured in gpsSerial.begin(...)

Parameter	Value in supplied implementation	Engineering interpretation
Wearable/relay LoRa serial rate	9,600 bit/s	Configured in Serial1 for both embedded programs
Desktop serial rate	9,600 bit/s	Matches LoRa USB receiver interface in app.py
Current GPS update timer	5,000 ms	Bench/debug interval in firmware; differs from project's intended 5-minute reporting interval
Heater threshold	25 °C	Prototype control threshold; not a validated survival/medical threshold
ACK token	"ok"	Simple textual acknowledgement
Packet format	ID-latitude,longitude	Current example uses ID 1 and four decimal places
Embedded queue capacity	5,000 String entries declared	Prototype choice; should be replaced by bounded ring buffer
Prediction history	Up to 5 wind-direction samples	Weighted circular average
Prediction target weights	70% movement / 20% wind average / 10% current wind	Heuristic, not learned
Prediction horizon	20 future markers	Visualization horizon; physical time horizon depends on sampling interval and step interpretation
Prediction step decay	0.995 per step	Heuristic gradual reduction in step length

Table 3.2. Key implementation parameters extracted from the supplied code

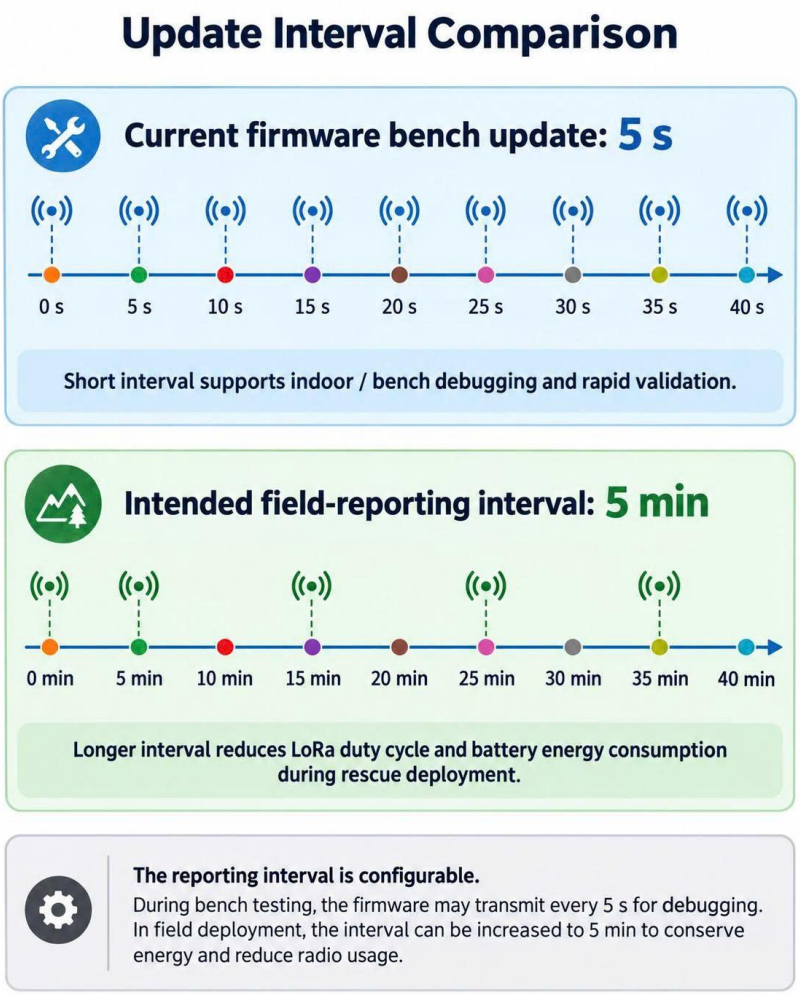


Figure 3.4. Bench-test and intended field-reporting timing concept

Figure interpretation. The project description specifies a five-minute position transmission cadence, while the current firmware uses a five-second GPS timer. A short interval is useful for debugging because it produces data quickly; a longer field interval can reduce radio duty cycle and energy use. The final chosen interval should be justified by rescue responsiveness and battery endurance.

3.11 Current Prototype Limitations

The current prototype is intentionally simple. It has a basic textual packet format, an acknowledgement token without packet sequence numbers, a relay mechanism without full mesh routing, no supplied link-budget or field-range measurements, no supplied battery-energy budget, and no quantified heating safety test. The desktop predictor uses wind and recent GPS movement but does not use measured ocean current, waves, a hydrodynamic model, or a trained machine-learning model. These limitations do not reduce the educational value of the prototype; they define the boundary between a strong student demonstration and a deployable maritime rescue product.

CHAPTER 4: PROTOTYPE TESTING AND VALIDATION

4.1 Validation Goals

Validation should answer a small number of practical engineering questions. Does the wearable node obtain a stable position and create correctly formatted packets? Does the radio protocol deliver and acknowledge packets? Can an intermediate relay forward unique packets without obvious duplication? Does the heater respond predictably to the sensor while remaining within safe temperature limits? Does the solar/battery system support the required duty cycle? Does the desktop software reconstruct the path and produce a drift estimate that is at least directionally useful in controlled tests?

4.2 Recommended Test Setup

Testing should progress from the bench to a controlled outdoor environment before any near-water or on-water demonstration. For radio tests, place the wearable node, relay, and receiver at known coordinates and record distance, antenna height, line-of-sight condition, packet count, acknowledgements, and radio signal metrics if the module exposes them. For thermal testing, use an independent thermometer or thermocouple near the heating surface. For power tests, use a current meter or USB/power analyzer appropriate to the supply. For prediction tests, record the actual later GPS track so the predicted path can be compared quantitatively.

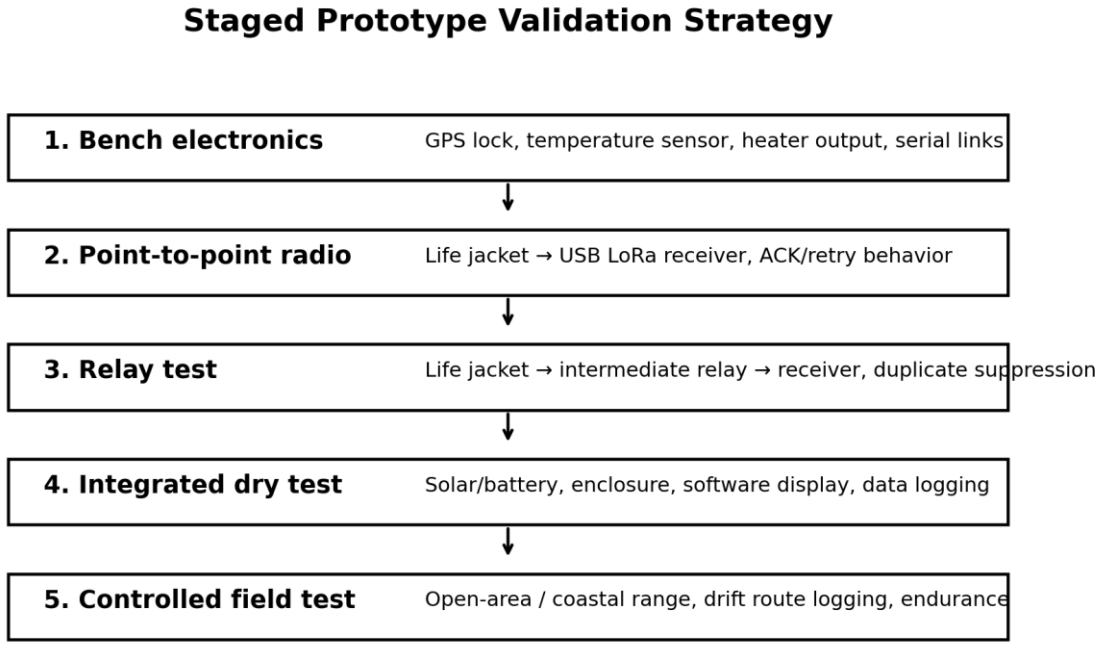


Figure 4.1. Staged prototype validation strategy

Figure interpretation. A staged approach keeps failures easy to diagnose and prevents an impressive field demonstration from hiding an unverified subsystem. Each stage should produce a small set of evidence files: logs, photographs, plots, and a short note describing the test conditions.

4.3 Validation Matrix

Test	Input / condition	Primary observation	Pass criterion for prototype
GPS stationary test	Device stationary beside trusted GPS/phone reference	Fix availability, coordinate stability, update timing	Consistent fixes; no malformed packets; error documented rather than assumed zero
GPS moving track	Walk/drive a known short route	Route continuity and packet order	Track follows route without unexplained jumps
Direct LoRa link	Wearable → receiver at increasing open-area distance	Packets sent, ACKs, losses	PDR and maximum demonstrated distance reported with conditions
Two-hop relay	Wearable → relay → receiver	Unique packet forwarding, duplicate behavior, latency	Packets reach receiver through relay and duplicates are bounded
Temperature threshold	Cool/warm sensor through threshold	Heater state versus measured temperature	Repeatable switching; fail-OFF on sensor fault in improved firmware
Power runtime	Defined duty cycle from fully charged battery	Current, voltage, runtime	Runtime is measured and compared with design target
Solar recharge	Measured sunlight window	Panel/charger current and battery recovery	Energy recovered is documented; no claim of autonomy without budget
Software replay	Feed a recorded packet file	Map points, ACK, route order, predicted markers	Correct parsing and repeatable prediction
Prediction accuracy	Use past GPS points to predict later track	Position/direction error	ADE/FDE or directional error reported for multiple trials

Table 4.1. Recommended prototype validation matrix

4.4 GPS Verification

A useful student-level GPS validation does not require expensive survey equipment. The student can place the device at several fixed outdoor points and compare its coordinates with a trusted smartphone or known map location, then collect a short moving track. Report the

number of successful fixes, observed update interval, and approximate scatter. The goal is to show that the end-to-end system uses real position data consistently, not to claim centimetre-level positioning.

4.5 Point-to-point LoRa Verification

The 8 km value should be converted from a design statement into a measured result. Begin with short ranges, then increase separation in a safe open area. At each distance send a fixed number of numbered packets and record how many are received and acknowledged. If the module provides RSSI or SNR, log those values as well. The strongest portfolio plot would show packet-delivery ratio versus distance together with test conditions and antenna height.

4.6 Relay-network Verification

For a two-hop demonstration, position the relay so that the rescue station can receive from the relay while the direct life-jacket-to-station link is intentionally degraded or blocked. Use numbered packets to prove that information arrived through the relay. Measure end-to-end latency and duplicates. Additional nodes can then be added one at a time. This produces much stronger evidence than simply multiplying a nominal 8 km range by the number of buoys.

4.7 Thermal and Power Verification

The heater should first be tested on a bench, not while worn. Place the temperature sensor and heating element in a controlled environment and log sensor temperature, independent reference temperature, heater state, current, and surface temperature. Verify that the output turns OFF when temperature rises above the threshold and that loss of sensor data cannot leave the heater continuously energized. Separately, measure average electronic current in idle, GPS acquisition, radio transmission, and heating states, then estimate runtime from actual battery capacity.

4.8 Software and Prediction Verification

The desktop application can be tested safely by replaying recorded serial packets. Verify that duplicates are ignored, acknowledgements are returned, points appear in the correct order, and the most recent position centres the map. For prediction, a simple quantitative method is to take the first part of a recorded moving track as history, generate a prediction, and compare predicted markers with later measured GPS points. Average displacement error (ADE), final displacement error (FDE), or heading error can be reported across multiple trials.

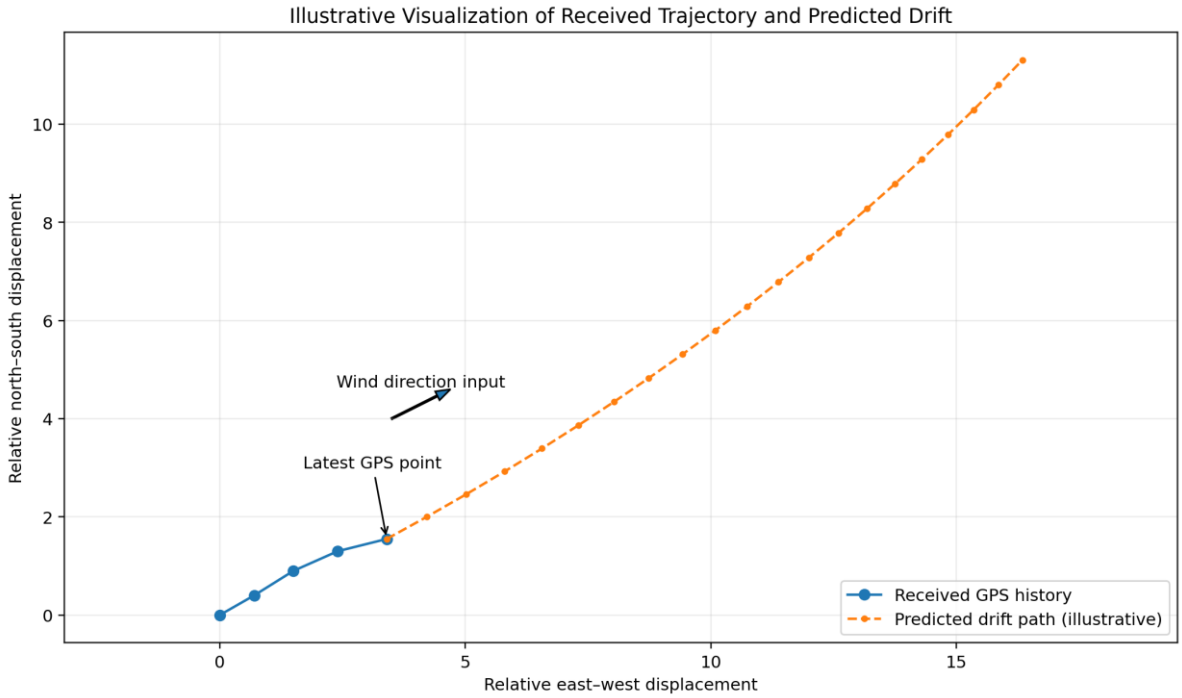


Figure 4.2. Illustrative received trajectory and drift prediction; not field-test data

Figure interpretation. The blue path represents received GPS history and the predicted path represents the software's future markers. This figure is intentionally illustrative and is not reported as experimental sea-trial data. A final portfolio can replace it with one actual recorded test and a prediction-versus-ground-truth comparison.

Metric	Definition / purpose	Why it matters
Packet Delivery Ratio (PDR)	Received unique packets / transmitted packets	Quantifies radio reliability instead of using a single success/failure anecdote
End-to-end latency	Time from life-jacket transmission to rescue-station receipt	Shows cost of relay forwarding and retry
Maximum demonstrated hop distance	Longest tested distance meeting selected PDR target	Grounds the range claim in evidence
Battery runtime	Time to voltage/SOC cutoff under defined duty cycle	Supports endurance claim
Solar energy recovered	Measured charging energy over defined conditions	Supports "solar-assisted" statement
Heater peak surface temperature	Maximum independently measured surface temperature	Essential thermal-safety evidence
Prediction ADE/FDE	Mean/final distance between predicted and later measured trajectory points	Quantifies whether prediction improves beyond a visual arrow

Metric	Definition / purpose	Why it matters
Last-fix age	Time since newest valid GPS packet	Prevents stale location from appearing current

Table 4.2. Quantitative metrics recommended for field testing

4.9 Evidence Available from the Current Project Package

The current package provides strong evidence of design and implementation but limited evidence of quantified field performance. The physical photograph demonstrates integration. The wearable firmware demonstrates GPS parsing, thermal control, buffering, radio transmission and acknowledgement logic. The relay firmware demonstrates duplicate checking and store-and-forward behavior. The Python application demonstrates serial reception, mapping, weather-data retrieval and a deterministic drift-prediction algorithm. No range-test log, battery-endurance dataset, solar charging dataset, thermal safety dataset, or prediction-accuracy dataset was supplied with the materials reviewed for this report.

4.10 Evidence Recommended for a College Portfolio

For U.S. college applications, the project is most compelling when the student shows ownership of the engineering process rather than presenting it as a finished commercial rescue system. A concise evidence package should show the problem, the built object, one piece of embedded code, one network test, one power or thermal measurement, and one software visualization. Every item should identify what the student personally designed, built, coded, tested, or improved.

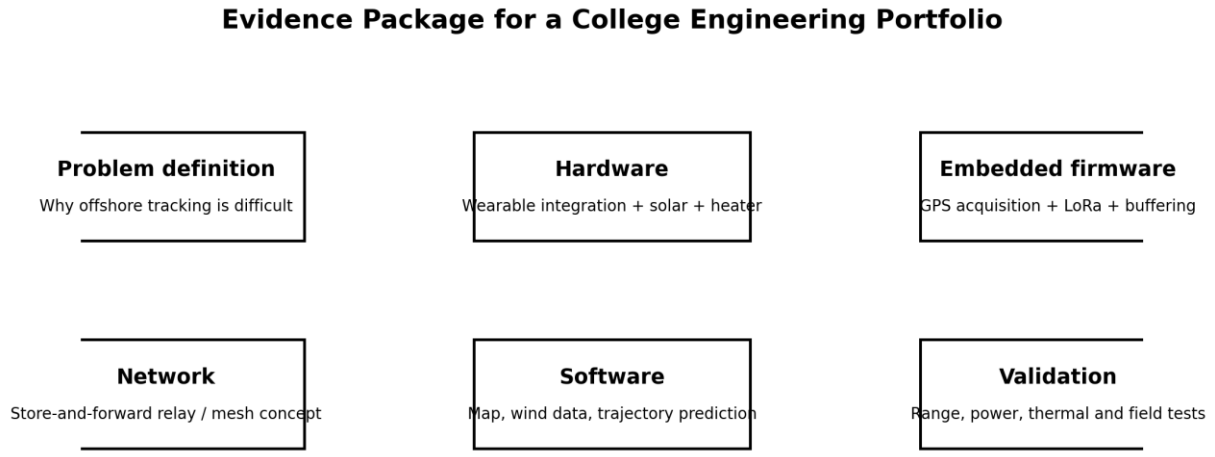


Figure 4.3. Recommended evidence package for a college engineering portfolio

Figure interpretation. Admissions readers do not need every source-code file. They need a coherent story with proof. Six well-chosen pieces of evidence can demonstrate systems thinking, embedded programming, communication engineering, software integration, and responsible validation.

CHAPTER 5: DISCUSSION, CONCLUSION AND FUTURE DEVELOPMENT

5.1 What the Prototype Demonstrates

The principal achievement is system integration. The project begins with a real rescue problem, converts it into engineering requirements, builds a wearable sensor and communication node, adds a relay concept, introduces energy harvesting and thermal support, and closes the information loop with rescue-station software. None of these individual technologies is new by itself, and the project does not need scientific novelty to be valuable. The educational value comes from making several disciplines work together and from being able to explain why each design choice exists.

The source package also shows iterative development. The wearable and relay firmware use straightforward embedded logic that a student can inspect and debug. The Python application progresses beyond a serial monitor to a graphical operator tool. The drift predictor, while heuristic, demonstrates an attempt to combine sensor history with external environmental data rather than showing only the latest GPS point.

5.2 Strengths and Engineering Trade-offs

Design choice	Strength	Trade-off / weakness
GPS-based tracking	Global, intuitive geographic output; easy to visualize	Requires sky visibility; fix quality and power consumption vary
LoRa low-rate radio	Appropriate for compact position packets and long-distance experiments	Range is highly environment-dependent and must be measured
Store-and-forward relay	Extends communication concept without cellular network	Current firmware is not a complete dynamic mesh protocol
Text packet + “ok” ACK	Easy to debug and teach	Needs sequence numbers, timestamps, integrity checking and bounded retries for robustness
Temperature-triggered heater	Demonstrates sensing-to-actuation integration	Wearable heating introduces serious thermal and power-safety requirements
Solar-assisted battery	Adds renewable-energy and endurance design dimension	Cannot guarantee autonomy without measured energy budget
Python GUI + weather API	Turns raw telemetry into an operator-facing system	Requires computer, map/data connectivity and improved logging for field use
Explainable drift heuristic	Transparent, lightweight, no training dataset needed	Not a learned AI model and not yet validated as an ocean-drift predictor

Table 5.1. Main strengths and engineering trade-offs

5.3 Limitations

The most important limitations are not hidden. The current software package does not demonstrate a trained AI model. The radio range and multi-hop coverage have not been supported by field data in the supplied materials. The relay code provides store-and-forward behavior but not full dynamic mesh routing. The thermal system lacks documented independent over-temperature protection. The solar subsystem lacks a measured energy budget. The position packet format lacks sequence numbers and timestamps, and the embedded queue uses dynamic Strings that are unsuitable for indefinite operation.

The prototype also should not be represented as a certified life-saving device. Real maritime equipment must satisfy requirements for waterproofing, buoyancy, battery safety, radio regulation, thermal protection, human factors, distress protocols, reliability, and environmental testing far beyond an educational prototype. Stating this clearly strengthens rather than weakens a college portfolio because it shows awareness of engineering responsibility.

5.4 Future Development

Future development should prioritize measurement and reliability before adding complexity. First, replace the text-only packet with a compact structured message containing device ID, sequence number, timestamp, latitude, longitude, temperature, battery state, and a checksum or CRC. Add retransmission limits and persistent logging. Second, upgrade the relay network with node IDs, hop count or TTL, link-quality information, and a documented forwarding rule. If a true mesh architecture is desired, implement and test route discovery or controlled flooding rather than relying on implicit forwarding.

Third, complete a power budget and redesign the heater as a safety-critical subsystem with independent cutoff protection. Fourth, quantify LoRa range and packet delivery in realistic open-area/coastal conditions. Fifth, turn the prediction module into an evidence-based comparison: keep the current transparent heuristic as a baseline, then train a model only after enough trajectories are collected. Candidate inputs could include recent GPS velocity, wind vector, measured/forecast surface current, wave direction, region, and time. The new model should be evaluated against the baseline using held-out trajectories and reported with ADE/FDE or equivalent metrics.

Finally, improve the physical portfolio evidence. A second-generation enclosure should document waterproofing method, connector selection, antenna placement, solar charge circuit, battery protection, and total mass. A one- to two-minute demonstration video can show the system acquiring GPS, transmitting through a relay, displaying the route on the computer, and triggering a controlled prediction. The student should narrate design decisions and limitations rather than only showing that the device “works.”

5.5 Conclusion

This project presents a student-built smart life-jacket prototype for offshore search-and-rescue support. The wearable node acquires GPS coordinates, buffers position packets, communicates over LoRa, and controls a heating output from a temperature measurement. A separate relay firmware demonstrates store-and-forward packet forwarding with duplicate

suppression and acknowledgements. At the rescue station, a Python application receives the data, displays the trajectory on a map, retrieves wind information, and generates a short-horizon drift estimate from recent motion and wind.

The report intentionally treats the system as a prototype rather than a scientific study or certified safety device. The most credible next step is not to make broader claims, but to collect better evidence: measured radio range, packet-delivery rate, relay latency, battery endurance, solar energy recovery, heater safety, and prediction error. With those additions, the project can become a particularly strong college engineering portfolio because it demonstrates embedded systems, wireless communication, energy integration, software development, data visualization, and responsible engineering judgment in one coherent application.

APPENDIX A: SOURCE-CODE TRACEABILITY

This appendix records how the supplied source files were interpreted for the report. It is intended to make the portfolio defensible: a reviewer can connect each claimed function to the implementation that supports it.

Source file	Function used in this report	Key evidence
ao/ao.ino	Wearable life-jacket embedded firmware	TinyGPS++ GPS parsing; DallasTemperature/OneWire sensor; heater pin 25; GPS Serial2 at 38,400; LoRa Serial1 at 9,600; 5,000 ms GPS timer; packet buffer; “ok” acknowledgement
phao-trunggian/phao-trunggian.ino	Intermediate LoRa relay firmware	Receive line; ignore “ok” as data; duplicate check; store unique packet; forward oldest entry; remove after acknowledgement
app-v2/app.py	Rescue-station desktop software	COM-port GUI; serial read thread; ACK; unique packet list; Tkinter map; Open-Meteo wind query; motion/wind heuristic prediction; 20 future markers
app-v2/gps.py	Development test for weather API	Retrieves current wind speed and direction from Open-Meteo for a fixed coordinate
app-v2/gyro.py and test.py	Auxiliary development files	Contain serial/3-D orientation and motion-processing experiments; not required by the core life-jacket communication workflow and therefore not used as evidence for central project claims

Table A.1. Source-code traceability to report functions

A.1 Current Wearable Packet Flow

```
GPS fix -> format '1-lat,lon' -> append to local queue
    -> transmit oldest packet over LoRa
    -> wait up to 1 s for 'ok'
    -> if ACK: delete oldest packet
    -> else: retain and retry
```

A.2 Current Relay Packet Flow

```
receive LoRa line
  -> if data packet and not duplicate: store
```

```
   -> if queue non-empty: forward oldest
   -> if ACK 'ok': delete oldest
   -> otherwise retain and continue listening
```

A.3 Current Prediction Logic

```
movement direction = angle(last GPS point - previous GPS point)
wind_avg = weighted circular mean(last <= 5 wind directions)
target = 0.7*movement + 0.2*wind_avg + 0.1*current_wind
turn_rate = min(0.3 + 0.04*wind_speed, 2.0)
for 20 steps:
  turn gradually toward target
  step *= 0.995
  create predicted map marker
```

APPENDIX B: FINAL DEMONSTRATION CHECKLIST

- Replace [STUDENT NAME] and [MENTOR NAME / TITLE] on both title pages.
- Record exact model numbers for GPS receiver, LoRa module, temperature sensor, solar panel, battery, charger/protection board, voltage regulators, heater, and USB LoRa receiver.
- Confirm legal/allowed radio frequency and transmit-power settings for the location where the prototype is demonstrated.
- Photograph the prototype from front, back, and inside the electronics enclosure at high resolution.
- Use numbered packets for the final radio test and save a CSV log with sent/received time, node ID, hop path if available, ACK status, RSSI/SNR if available, and coordinates.
- Measure direct-link packet delivery at several distances. Report the longest demonstrated range; do not substitute the nominal radio specification.
- Demonstrate at least one two-hop relay path in which the relay is necessary for delivery.
- Measure battery runtime under a clearly defined duty cycle and record solar charging current/energy under defined lighting conditions.
- Bench-test the heater with an independent temperature measurement and a fail-OFF safety method before any human-worn active test.
- Record a GPS route and compare one prediction generated from early points with the later measured path.
- Prepare a one- to two-minute portfolio video showing: prototype overview -> GPS acquisition -> LoRa packet -> relay -> desktop map -> prediction -> measured test evidence.
- In the video and application essay, state exactly what the student personally designed, assembled, coded, tested, and debugged.
- Do not call the current predictor a trained machine-learning model unless a trained model is actually added and evaluated.
- Do not present the prototype as a certified maritime rescue device or substitute for approved emergency equipment.

APPENDIX C: COLLEGE-APPLICATION PORTFOLIO SUMMARY

C.1 One-paragraph Project Summary

Designed and built a smart life-jacket engineering prototype that combines GPS tracking, LoRa long-range communication, store-and-forward relay nodes, temperature-triggered heating, solar-assisted lithium power, and a Python rescue-station application. The embedded system buffers position packets and retransmits them until acknowledged, while intermediate relay nodes can receive, de-duplicate, and forward data toward rescuers. The desktop application maps the received route, retrieves local wind information, and generates an explainable short-term drift estimate from recent GPS motion and wind. The project is presented as an application prototype with explicit plans for radio-range, power, thermal-safety, and prediction-accuracy validation.

C.2 CV-ready Project Entry

Suggested CV wording. Smart Life Jacket for Offshore Search and Rescue — Student Engineering Prototype. Integrated GPS tracking, LoRa store-and-forward relay communication, temperature-triggered heating, solar-assisted battery power, and a Python mapping application with wind-informed drift prediction. Developed embedded packet buffering/ACK logic and a relay-forwarding workflow; planned validation for radio range, packet delivery, power endurance, thermal safety, and trajectory prediction.

C.3 Skills Demonstrated

- Embedded C/C++ and microcontroller firmware
- GPS / GNSS data acquisition and serial interfaces
- LoRa low-data-rate wireless communication
- Store-and-forward relay-network logic
- Python GUI development and serial communication
- Map visualization and API integration
- Rule-based trajectory prediction and data fusion
- Solar/battery system integration and power-budget thinking
- Prototype integration, debugging, validation planning, and engineering documentation

C.4 Student Contribution Statement

For admissions use, replace this paragraph with a factual first-person statement. Example structure: “I was responsible for [hardware integration], [embedded firmware modules], [relay-network code], [desktop application], and [specific tests]. My mentor supported me by [scope of guidance]. The part I found most difficult was [technical problem], and I solved it by [specific debugging/design action].” Do not claim work performed primarily by a mentor or teammate.